

I Autour de la dichotomie (E3A 2019)

Voici une tentative de code d'une fonction C pour tester par dichotomie si un entier e se trouve dans un tableau d'entiers T :

```
bool dichotomie(int e, int* T, int n) {
    // Précondition : n >= 1 et T est un tableau de n entiers trié
    // dans l'ordre croissant
    int g = 0, d = n - 1;
    while (d - g > 0) {
        int m = (g + d) / 2;
        if (T[m] >= e) d = m;
        else g = m;
    }
    return T[g] == e;
}
```

1. Pour quelle raison ne remplace-t-on pas la précondition par un appel à une fonction qui trierait T dans l'ordre croissant ?

Solution : Cela augmenterait la complexité : elle passerait de $O(\log n)$ à $O(n \log n)$ si l'on utilisait, par exemple, un tri en $O(n \log n)$. Le fait que le tableau soit déjà trié ne garantit d'ailleurs pas que l'algorithme de tri choisi s'exécute en temps linéaire.

2. Montrer que la fonction `dichotomie` ne termine pas forcément.

Solution : Sur le tableau $T = \{0, 1\}$, l'appel `dichotomie(1, T, 2)` ne termine pas : on a indéfiniment $g = 0$, $d = 1$ et $m = 0$, donc l'affectation $g = m$ ne change pas l'état de la boucle.

3. Indiquer sans justification la ou les corrections à apporter pour que la fonction `dichotomie` termine, tout en restant correcte.

Solution : Il suffit de remplacer $g = m$ par $g = m + 1$. En effet, si $T[m] < e$, on peut exclure l'indice m de la zone de recherche.

4. Justifier que la fonction corrigée termine et est correcte.

Solution : Montrons alors que $d - g$ décroît strictement à chaque itération du `while`.

Considérons une itération du `while` :

- Si $T[m] \geq e$: on remplace d par m et $m - g = \left\lfloor \frac{g+d}{2} \right\rfloor - g \leq \frac{g+d}{2} - g = \frac{g+d-2g}{2} = \frac{d-g}{2} < d-g$ car $d-g > 0$ (condition du `while`).
- Si $T[m] < e$: on remplace g par $m+1$ et $d - (m+1) = d - \left(\left\lfloor \frac{g+d}{2} \right\rfloor + 1 \right) < d - \frac{g+d}{2}$ (car $\left\lfloor \frac{g+d}{2} \right\rfloor > \frac{g+d}{2} - 1$).

$$\text{D'où } d - (m+1) < \frac{d-g}{2} < d-g.$$

Tant que la boucle est exécutée, $d - g$ est donc un entier naturel qui décroît strictement. Il finit par être nul, ce qui prouve la terminaison.

Pour prouver la correction, considérons l'invariant suivant : « e appartient au tableau initial si et seulement s'il appartient au sous-tableau formé des éléments de T d'indices g à d ». Il est vrai avant la première itération. Si $T[m] \geq e$, tous les éléments d'indice strictement supérieur à m sont supérieurs ou égaux à $T[m]$, donc ils ne peuvent pas être égaux à e , sauf si $T[m] = e$, auquel cas l'indice m est conservé : on peut donc remplacer d par m . Si $T[m] < e$, les éléments d'indices inférieurs ou égaux à m sont strictement inférieurs à e , donc on peut remplacer g par $m+1$. L'invariant est ainsi préservé.

À la sortie, $g = d$. L'entier e appartient donc au tableau si et seulement si $T[g] == e$. Ainsi, la fonction `dichotomie` est correcte.

Une première extension consiste à réduire le problème non pas en 2 mais en 3 : c'est le principe de trichotomie.

5. Écrire une fonction `bool trichotomie(int e, int* T, int n)` qui renvoie `true` si l'élément e se trouve dans T et `false` sinon.

Solution : On choisit deux indices $m_1 = \lfloor (2g+d)/3 \rfloor$ et $m_2 = \lfloor (g+2d)/3 \rfloor$, qui partagent l'intervalle de recherche en trois parties de tailles voisines :

```

bool tricho(int e, int* T, int n) {
    int g = 0, d = n - 1;
    while (g < d) {
        int m1 = (2*g + d) / 3;
        int m2 = (g + 2*d) / 3;
        if (e <= T[m1]) {
            d = m1;
        } else if (e <= T[m2]) {
            g = m1 + 1;
            d = m2;
        } else {
            g = m2 + 1;
        }
    }
    return T[g] == e;
}

```

6. Estimer la complexité de `tricho(e, T, n)` en fonction de `n`. Comparer avec la méthode par dichotomie.

Solution : À chaque itération, la taille de la zone de recherche est divisée approximativement par trois. Le nombre d'itérations est donc en $O(\log_3 n)$, c'est-à-dire en $O(\log n)$. La complexité asymptotique est la même que pour la dichotomie.

Si l'on compte uniquement les comparaisons entre `e` et un élément du tableau, une itération en effectue jusqu'à deux. Dans le pire cas, la trichotomie utilise donc environ $2\log_3 n$, contre $\log_2 n$ pour la dichotomie ; elle ne constitue pas une amélioration dans ce modèle.

Une seconde extension consiste à adapter le principe de dichotomie au cas d'une matrice d'entiers dont les éléments sont triés en colonne de haut en bas et de gauche à droite. L'illustration ci-dessous indique l'ordre des éléments d'une matrice à 4 lignes et 6 colonnes :

$$\begin{pmatrix} 0 & 4 & 8 & 12 & 16 & 20 \\ 1 & 5 & 9 & 13 & 17 & 21 \\ 2 & 6 & 10 & 14 & 18 & 22 \\ 3 & 7 & 11 & 15 & 19 & 23 \end{pmatrix}$$

7. Expliquer comment déterminer si un entier `e` se trouve dans une telle matrice de taille $n \times p$, en complexité logarithmique en $\max(n, p)$.

Solution : On effectue une première recherche dichotomique parmi les derniers éléments des colonnes afin de trouver la première colonne dont le dernier élément est supérieur ou égal à `e`. On effectue ensuite une seconde recherche dichotomique dans cette colonne.

La complexité vaut $O(\log p) + O(\log n) = O(\log(\max(n, p)))$.

II Conversion d'arbre binaire de recherche en tas

On définit un arbre binaire par : `type 'a arb = V | N of 'a * 'a arb * 'a arb;;`

1. Rappeler la définition d'une file de priorité min, d'un tas min et d'une implémentation par un tableau. Le tableau est-il forcément trié ?

Solution : Une file de priorité min est une structure de données qui permet d'insérer des éléments et d'extraire le minimum. Un tas min est un arbre binaire complet dont chaque sommet est inférieur ou égal à ses fils. Il peut être implémenté par un tableau : le sommet racine est à l'indice 0, le fils gauche d'un sommet à l'indice i est à l'indice $2i + 1$ et le fils droit à l'indice $2i + 2$.

2. Écrire une fonction `infixe : 'a arb -> 'a list`, si possible en complexité linéaire, renvoyant la liste des sommets d'un arbre parcourus dans l'ordre infixe.

Solution : On utilise un accumulateur pour éviter l'opérateur `@`, qui conduirait à une complexité quadratique dans le

pire cas. La fonction auxiliaire effectue un ajout en tête par sommet, d'où une complexité linéaire.

```
let infixe a =  
  let rec aux a acc =  
    match a with  
    | V -> acc  
    | N(e, g, d) -> aux g (e :: aux d acc)  
  in  
  aux a []
```

3. Rappeler la définition d'un arbre binaire de recherche (ABR). Montrer qu'un arbre binaire est un ABR si et seulement si son parcours infixe est croissant.

Solution : Un arbre binaire est un ABR si, pour chaque sommet, toutes les étiquettes de son sous-arbre gauche sont strictement inférieures à la sienne et toutes les étiquettes de son sous-arbre droit lui sont strictement supérieures.

Montrons d'abord par induction structurelle que le parcours infixe de tout ABR est strictement croissant.

- L'arbre vide a pour parcours infixe la liste vide, qui est croissante.
- Soit $N(e, g, d)$ un ABR. Les arbres g et d sont eux-mêmes des ABR ; par hypothèse d'induction, leurs parcours infixes sont croissants. De plus, toute étiquette de g est strictement inférieure à e , et toute étiquette de d est strictement supérieure à e . La concaténation du parcours infixe de g , de $[e]$ et du parcours infixe de d est donc strictement croissante : c'est précisément le parcours infixe de $N(e, g, d)$.

Montrons réciproquement, également par induction structurelle, que tout arbre dont le parcours infixe est strictement croissant est un ABR.

- L'arbre vide est un ABR.
- Soit $N(e, g, d)$ un arbre dont le parcours infixe est strictement croissant. Les parcours infixes de g et de d , qui en sont des sous-listes contiguës, sont eux-mêmes strictement croissants. Par hypothèse d'induction, g et d sont donc des ABR. Par ailleurs, les étiquettes de g apparaissent toutes avant e dans le parcours : elles lui sont donc strictement inférieures. De même, les étiquettes de d apparaissent toutes après e et lui sont donc strictement supérieures. Ainsi, $N(e, g, d)$ est un ABR.

Les deux implications sont démontrées.

4. Écrire une fonction `tas_of_abr` : `'a arb -> 'a array` telle que, si a est un arbre binaire de recherche à n éléments, `tas_of_abr a` renvoie, en $O(n)$, un tableau représentant un tas min avec les mêmes éléments que a .

Solution : Le parcours infixe d'un ABR est croissant. Le tableau obtenu représente donc un tas min : pour chaque indice valide, on a $t.(i) \leq t.(2*i+1)$ et $t.(i) \leq t.(2*i+2)$. Les deux conversions étant linéaires, la complexité totale est en $O(n)$.

```
let tas_of_abr a =  
  let l = infixe a in  
  let t = Array.make (List.length l) 0 in  
  let rec aux i = function  
    | [] -> t  
    | e :: r -> t.(i) <- e; aux (i + 1) r  
  in  
  aux 0 l
```

5. Comment peut-on implémenter les fonctions `take` (pour renvoyer et supprimer la racine d'un tas) et `add` (pour ajouter un élément à un tas) sur un tas min représenté par un tableau ? Avec quelle complexité ?

Solution : La fonction `take` renvoie l'élément à l'indice 0 du tableau, le remplace par le dernier élément du tableau et l'échange tant que nécessaire avec son enfant le plus petit pour restaurer la propriété de tas. La fonction `insert` ajoute un élément à la fin du tableau et l'échange tant que nécessaire avec son parent pour restaurer la propriété de tas. Les deux fonctions ont une complexité en $O(\log n)$.

6. En déduire une fonction `abr_of_tas` : `'a array -> 'a arb` en $O(n \log n)$ telle que, si t représente un tas min à n éléments, `abr_of_tas t` renvoie un arbre binaire de recherche presque complet et avec les mêmes éléments que t .

Solution : On obtient ainsi en $O(n \log n)$ un tableau trié. On construit ensuite en $O(n)$ un ABR presque complet en choisissant récursivement l'élément médian de chaque sous-tableau comme racine.

```

let abr_of_tas t =
  let n = Array.length t in
  let tab = Array.init n (fun i -> take t) in
  let rec aux i j =
    (* Convertit tab.(i), ..., tab.(j - 1) en ABR *)
    if i >= j then V
    else
      let m = (i + j) / 2 in
      N(tab.(m), aux i m, aux (m + 1) j)
  in
  aux 0 n

```

Montrons par récurrence forte sur q que l'arbre construit à partir de q éléments est presque complet et de hauteur $\lfloor \log_2 q \rfloor$ lorsque $q \geq 1$. Le résultat est immédiat pour $q = 1$. Pour $q \geq 2$, posons $h = \lfloor \log_2 q \rfloor$; on a donc $2^h \leq q < 2^{h+1}$. Les sous-arbres gauche et droit contiennent respectivement

$$q_g = \left\lfloor \frac{q}{2} \right\rfloor \quad \text{et} \quad q_d = q - 1 - q_g = \left\lfloor \frac{q-1}{2} \right\rfloor$$

éléments. Ainsi, $2^{h-1} \leq q_g < 2^h$ et $2^{h-1} - 1 \leq q_d < 2^h$. Par hypothèse de récurrence, les deux sous-arbres possèdent donc tous leurs niveaux jusqu'au niveau $h-2$, et le sous-arbre gauche possède au moins un sommet au niveau $h-1$. Dans l'arbre obtenu, les niveaux 0 à $h-1$ sont par conséquent complets, tandis que le niveau h peut ne pas l'être. L'arbre est donc presque complet et sa hauteur vaut $h = \lfloor \log_2 q \rfloor$, donc est en $O(\log q)$.

7. Peut-on faire mieux que $O(n \log n)$ pour la question précédente ?

Solution : Non, dans le modèle des tris par comparaisons. Si la conversion était en $o(n \log n)$, on pourrait trier un tableau $\mathbf{t}$ de taille n en $o(n \log n)$:

- (i) transformer $\mathbf{t}$ en tas min, avec l'algorithme linéaire du cours, en $O(n)$;
- (ii) convertir ce tas en ABR, en $o(n \log n)$;
- (iii) renvoyer le parcours infixe de cet ABR, en $O(n)$.

Cela contredirait la borne inférieure en $\Omega(n \log n)$ du tri par comparaisons.

III Plus courts chemins et Dijkstra

On considère un graphe orienté dont les sommets sont les entiers de 0 à 5. Chaque arc est muni d'un poids entier strictement positif. Le graphe est représenté par listes d'adjacence : une paire $(\mathbf{v}, \mathbf{p})$ représente un arc vers $\mathbf{v}$ de poids $\mathbf{p}$.

```

let g = [
  [(1, 4); (2, 2)];           (* sommet 0 *)
  [(3, 5)];                   (* sommet 1 *)
  [(1, 1); (3, 4); (4, 5)];  (* sommet 2 *)
  [(4, 1); (5, 3)];          (* sommet 3 *)
  [(5, 1)];                   (* sommet 4 *)
  []                           (* sommet 5 *)
]

```

On note $\delta(s, v)$ la longueur d'un plus court chemin du sommet s au sommet v , avec $+\infty$ lorsqu'il n'existe aucun tel chemin.

1. Exécuter à la main l'algorithme de Dijkstra à partir du sommet 0. Donner l'ordre dans lequel les sommets deviennent vus, les distances obtenues et un plus court chemin de 0 à 5.

Solution : Les sommets deviennent vus dans l'ordre

0, 2, 1, 3, 4, 5.

Les distances depuis 0, dans l'ordre des sommets de 0 à 5, sont

0, 3, 2, 6, 7, 8.

Par exemple, $0 \rightarrow 2 \rightarrow 4 \rightarrow 5$ est un plus court chemin de 0 à 5, de longueur $2 + 5 + 1 = 8$.

2. Énoncer un invariant de l'algorithme de Dijkstra et démontrer sa correction.

Solution : On maintient un ensemble V de sommets vus et une estimation $d[v]$ pour chaque sommet v . L'invariant est le suivant :

- pour tout $v \in V$, $d[v] = \delta(s, v)$;
- pour tout $v \notin V$, $d[v]$ est la longueur minimale d'un chemin de s à v dont tous les sommets, sauf éventuellement v , appartiennent à V , ou $+\infty$ si un tel chemin n'existe pas.

Il est vrai initialement avec $V = \emptyset$, $d[s] = 0$ et $d[v] = +\infty$ pour $v \neq s$. Supposons-le vrai et soit $u \notin V$ un sommet d'estimation minimale. Considérons un plus court chemin de s à u et le premier sommet y de ce chemin qui n'appartient pas à V ; son prédécesseur x appartient à V . Lorsque x est devenu vu, l'estimation associée à l'arc (x, y) a été mise à jour, donc

$$d[y] \leq \delta(s, x) + w(x, y) = \delta(s, y) \leq \delta(s, u).$$

Comme u a une estimation minimale, $d[u] \leq d[y]$. Par ailleurs, $d[u]$ est la longueur d'un chemin de s à u , donc $d[u] \geq \delta(s, u)$. Toutes ces inégalités imposent $d[u] = \delta(s, u)$: on peut ajouter u à V . La mise à jour des estimations de ses voisins restaure ensuite la seconde partie de l'invariant. À la fin, toutes les distances accessibles sont donc correctes.

3. Écrire une fonction `dijkstra : (int * int) list array -> int -> int array` qui renvoie les distances depuis une source. On utilisera `max_int` pour représenter $+\infty$ et on cherchera naïvement, à chaque étape, le sommet non vu d'estimation minimale.

Solution :

```
let dijkstra g s =
  let n = Array.length g in
  let dist = Array.make n max_int in
  let vus = Array.make n false in
  dist.(s) <- 0;
  for _k = 0 to n - 1 do
    let u = ref (-1) in
    for v = 0 to n - 1 do
      if not vus.(v) && dist.(v) <> max_int
        && (!u = -1 || dist.(v) < dist.(!u)) then
        u := v
    done;
    if !u <> -1 then begin
      vus.(!u) <- true;
      List.iter (fun (v, poids) ->
        let nd = dist.(!u) + poids in
        if not vus.(v) && nd < dist.(v) then
          dist.(v) <- nd
      ) g.(!u)
    end
  done;
  dist
```

Le test `dist.(v) <> max_int` évite notamment d'ajouter un poids à `max_int`.

4. Déterminer la complexité de cette implémentation pour un graphe à n sommets et m arcs. Comment l'améliorer pour un graphe peu dense ?

Solution : La recherche linéaire du minimum coûte $O(n)$ et est répétée n fois, soit $O(n^2)$. Chaque arc est examiné au plus une fois, ce qui ajoute $O(m)$. La complexité totale est donc $O(n^2 + m) = O(n^2)$. En utilisant une file de priorité min représentée par un tas, on obtient une complexité en $O((n + m) \log n)$, souvent écrite $O(m \log n)$ lorsque tous les

sommets sont accessibles et $m \geq n - 1$.

5. On souhaite également compter le nombre de plus courts chemins de s à chaque sommet. Expliquer comment modifier les mises à jour, puis donner ces nombres pour le graphe de l'énoncé avec $s = 0$.

Solution : On ajoute un tableau **nb**, initialisé à 0, sauf **nb**.(**s**) = 1. Lors de l'examen d'un arc de **u** vers **v**, on effectue :

```
if nd < dist.(v) then begin
  dist.(v) <- nd;
  nb.(v) <- nb.(!u)
end else if nd = dist.(v) then
  nb.(v) <- nb.(v) + nb.(!u)
```

Les poids étant strictement positifs, tous les prédécesseurs possibles de **v** sur un plus court chemin ont une distance strictement inférieure à celle de **v** et deviennent donc vus avant lui. Dans l'ordre des sommets de 0 à 5, les nombres de plus courts chemins sont

1, 1, 1, 1, 2, 2.

Les deux plus courts chemins de 0 à 5 sont $0 \rightarrow 2 \rightarrow 4 \rightarrow 5$ et $0 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$.

6. Pourquoi l'hypothèse de positivité des poids est-elle importante ? Donner un exemple où Dijkstra échoue en présence d'un poids négatif et indiquer un algorithme qui fonctionne dans ce cas.

Solution : Pour la correction de Dijkstra, des poids positifs ou nuls suffisent : c'est l'absence de poids négatifs qui est essentielle. La preuve suppose qu'une fois un sommet u d'estimation minimale devenu vu, aucun chemin passant par un sommet non vu ne peut améliorer sa distance. Cette propriété devient fausse avec des poids négatifs. Par exemple, considérons les arcs $0 \rightarrow 1$ de poids 2, $0 \rightarrow 2$ de poids 5 et $2 \rightarrow 1$ de poids -10 . Dijkstra marque le sommet 1 comme vu avec la distance 2 avant d'examiner le sommet 2, alors que le chemin $0 \rightarrow 2 \rightarrow 1$ a pour longueur -5 . L'algorithme de Floyd-Warshall peut être utilisé lorsque des poids négatifs sont autorisés. La stricte positivité demandée dans l'énoncé garantit en outre que le comptage de la question précédente peut être effectué lors de la mise à jour des estimations.